

Journey to a Well Governed Cloud

Mike Benkovich

Imagine! Technologies, Inc.

mike@benko.com | [@mbenko](https://twitter.com/mbenko) | www.benkotips.com

Mike Benkovich

- Enterprise Cloud Architect & **Consultant**
- Live in **Minneapolis**
- Founder of **Imagine Technologies**, Inc.
- Developing Courses for **LinkedIn** Learning
- Blog www.benkoTIPS.com
- Follow **@mbenko** on **Twitter**
- Send me **Feedback!** mike@benko.com
- Azure **Office Hours** on Fridays! <https://bit.ly/BnkAzHrs>

Mike Benkovich

Enterprise Cloud Architect,
Consultant, Developer Tools Ev...



Azure Office Hour Fridays



@MikeBenkovich



05/04/2021



BenkoTIPS

Thinking about going to Cloud? I've been consulting around Azure for the last 8 years since I left Microsoft where I helped launch it in 2009. I want to offer my support, so I'm starting a thing called Azure Office Hours on Fridays, where anyone can block out 15 minutes to chat about anything Azure.

1. Find a time that works - <https://bit.ly/BnkAzHrs>
2. Let me know what you want to talk about
3. Let's chat!

I speak at conferences and have LinkedIn learning courses on Azure and DevOps including templating, compute, storage, messaging, networking and governance topics.

My [calendar](#) is open. Let's connect!



0



Comments



My sessions ... Cloud & AI Summit

❖ Taking the Blues out of Azure: Well Governed Cloud

Thursday 3:00-4:30 – *Discovery Ballroom C*

❖ From GPT to Squad: AI Developer Tools

Friday 10:15-11:45 – *Achievement*

❖ Infrastructure as Code: ARM/Bicep, Terraform, Pulumi

Friday 1:15-2:30 – *Discovery Ballroom A*

Today

How to go from **Idea** to **Cloud** App

Discover **Visual Studio** tools to build **Connected** Apps

Use **Infrastructure asCode** to manage cloud resources!

Enable Continuous **Value** with **DevOps**

Blueprints create **Landing Zones** with **Cloud Governance**

TODAY

Building myDadApp
A Well Governed Cloud
Creating Landing Zones

My Dad App

What is the "Cloud Journey"

An **Application** is an Idea

Cloud Application



Data

Code

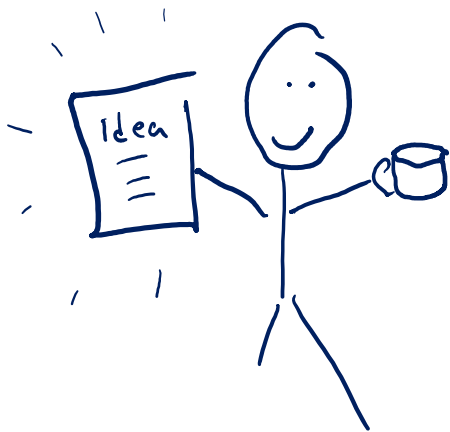
+ Infrastructure

= Application



- Runs in a cloud datacenter
- Managed on virtual hardware
- Monitored
- Configurable
- Scalable

I've got an idea!



My kids graduating...

Need guidance!

- Hard to be everywhere
- Words of wisdom help

Mobile Friendly

Reuse API Economy?

The Dad App

Web Application

- Mobile friendly
- ASP.NET Core Web App
- Calls API for Joke

Configuration

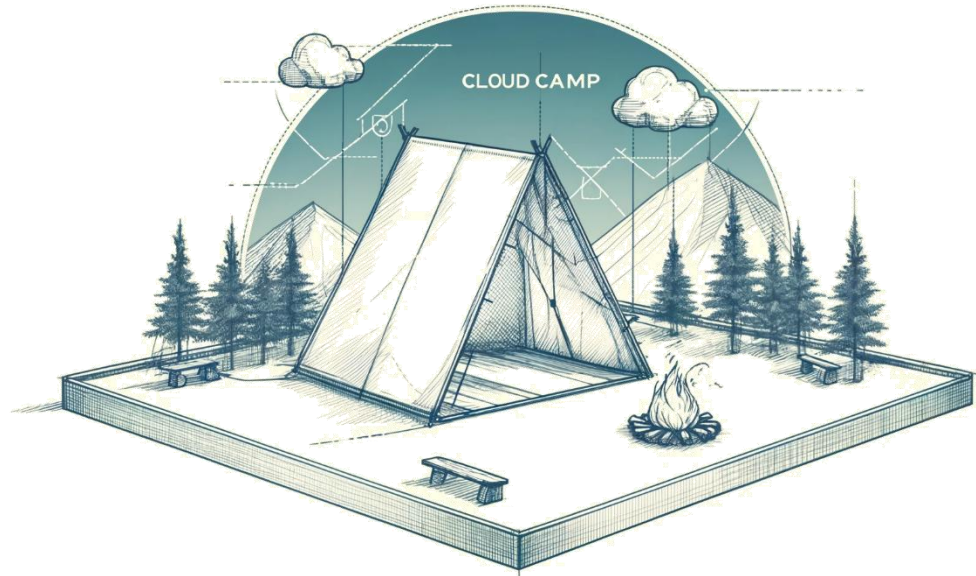
Monitored

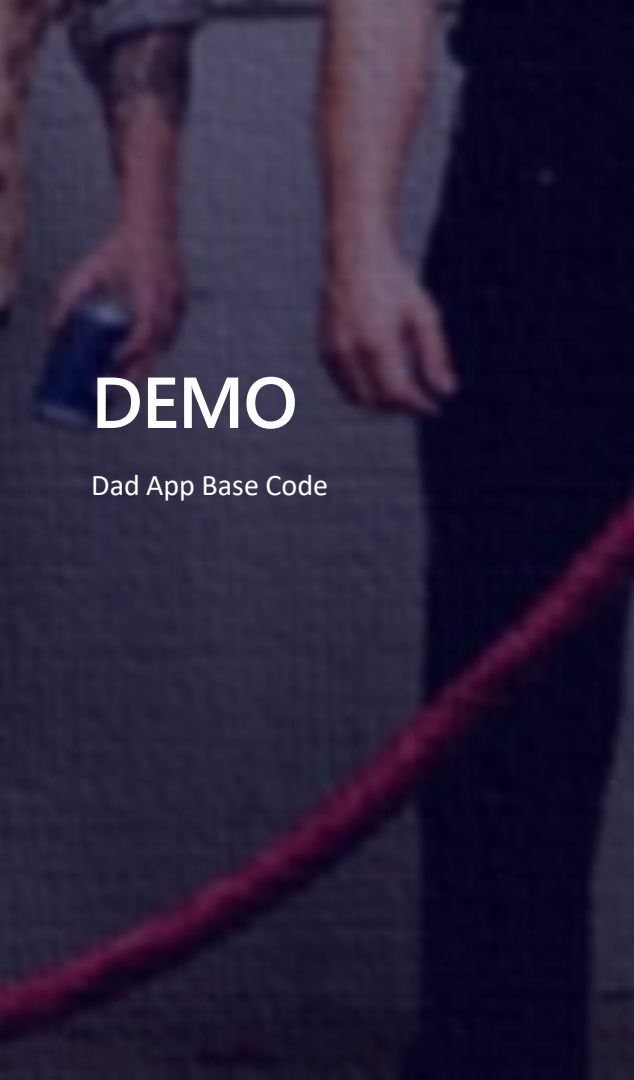
Keep it simple!

- Use existing tools
- Click to deploy
- Manage in Portal



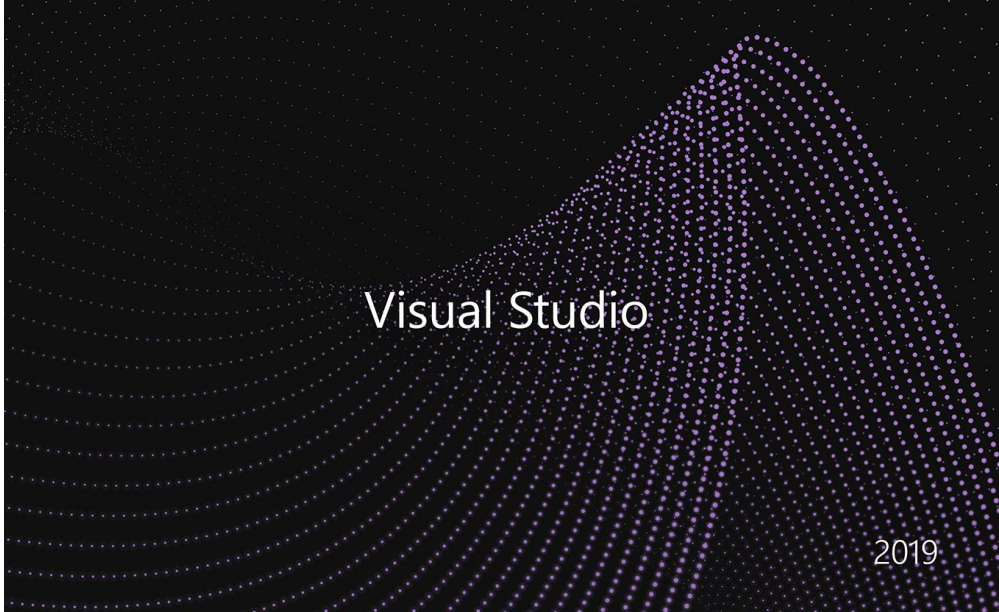
Azure Boot Camp





DEMO

Dad App Base Code



Visual Studio

2019

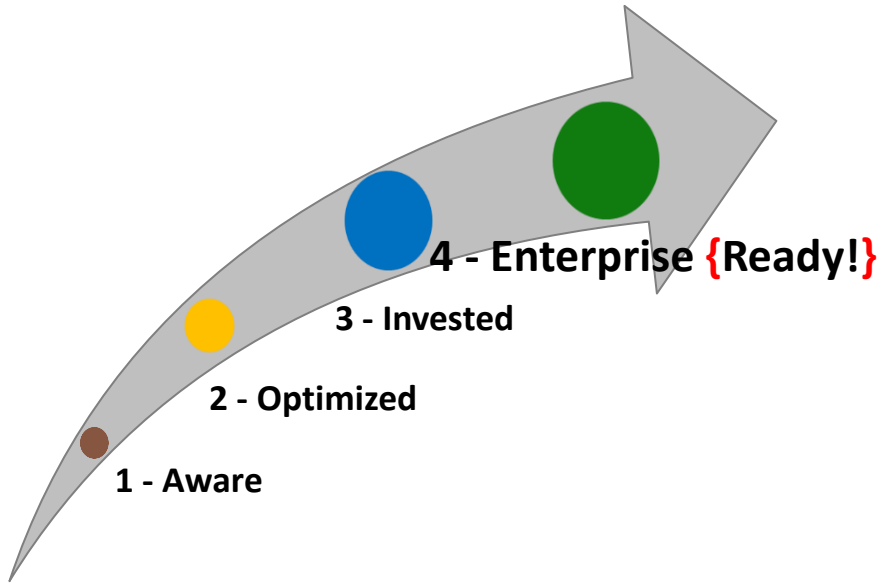
Doing things right

is not the same as

Doing the right things

- Steve Jobs?

Cloud Journey



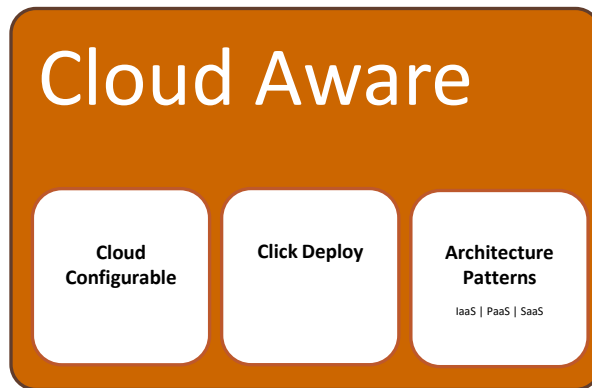
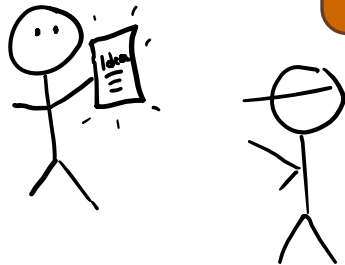
Step 1 – The Cloud Aware App

Features

- From Visual Studio
- Runs in the cloud
- Existing code, minimal changes
- Config settings in portal

Concerns

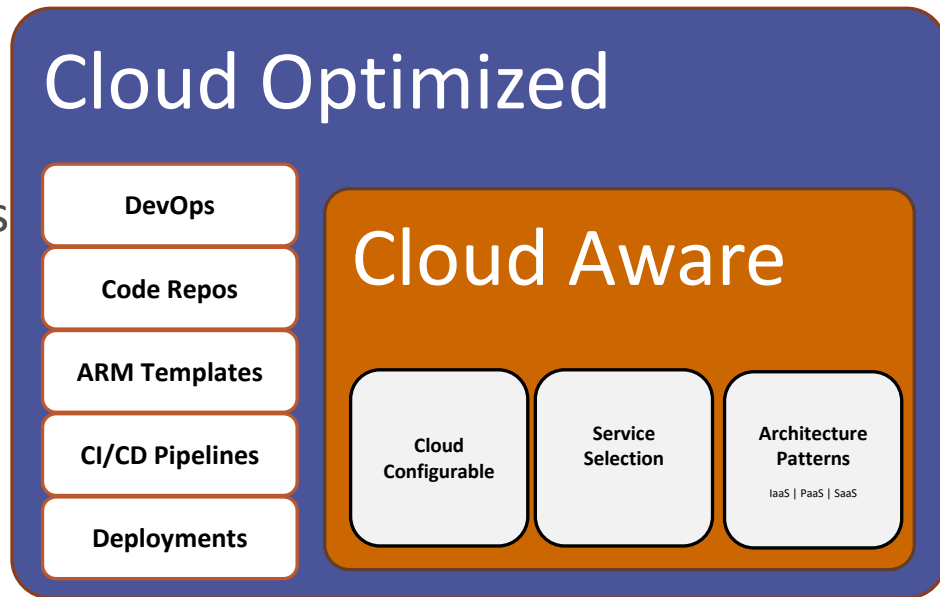
- Repeatability
- Click to deploy
- Governance concerns



Step 2 - Cloud Optimized

Requirements

- Code collaboration
- DevOps Processes
- Cloud service capabilities
- Application Lifecycles
- Managing delivery



Step 3 – Cloud Invested

Governance

Operational

Dashboards

Cost Mgmt

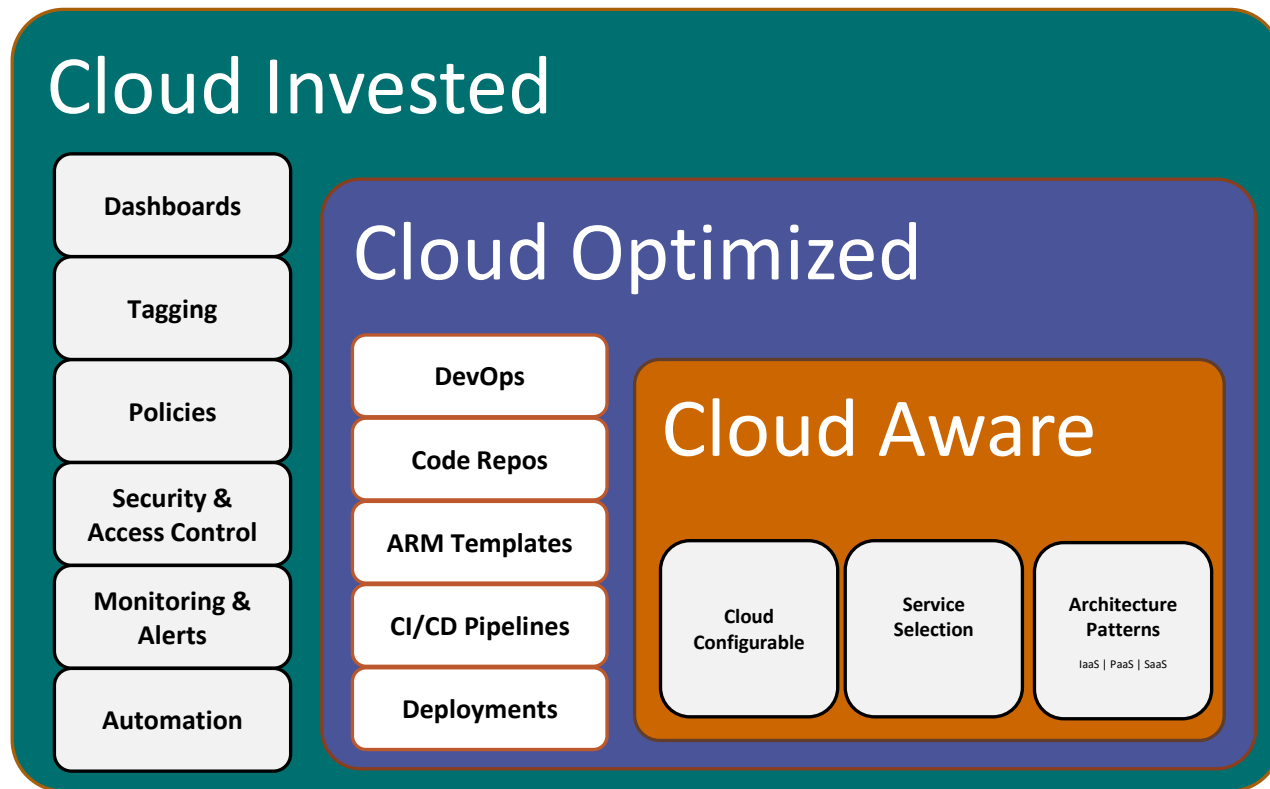
Monitoring

Blueprints

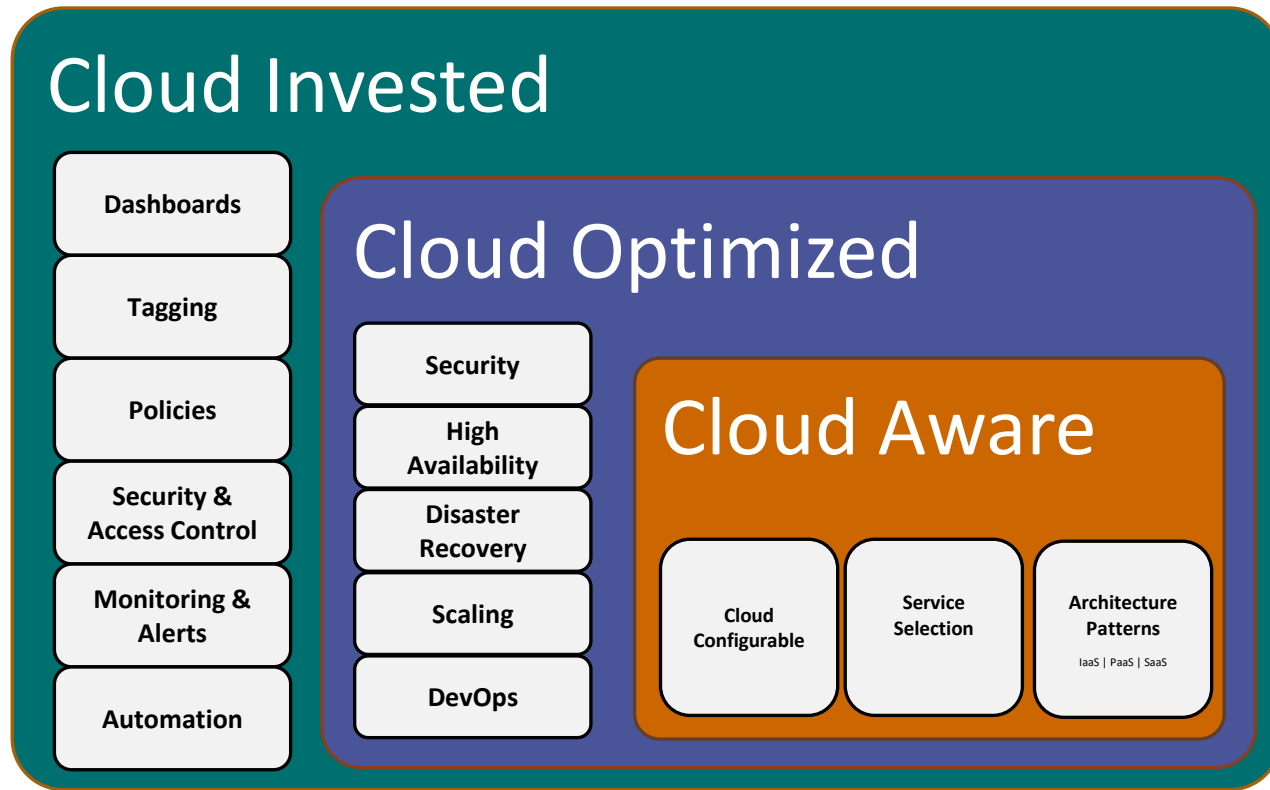
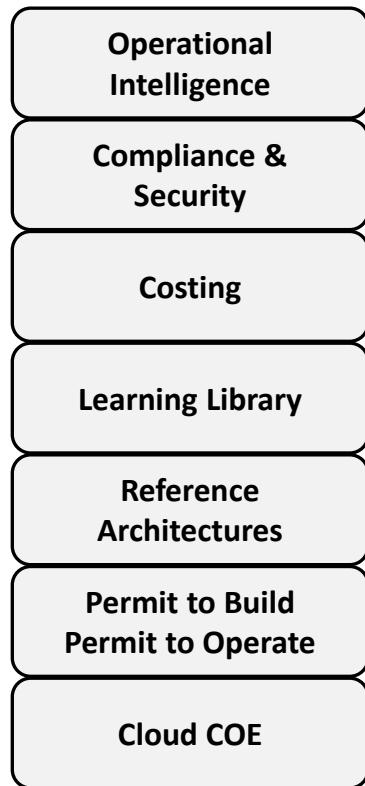
Access

Compliance

Policies

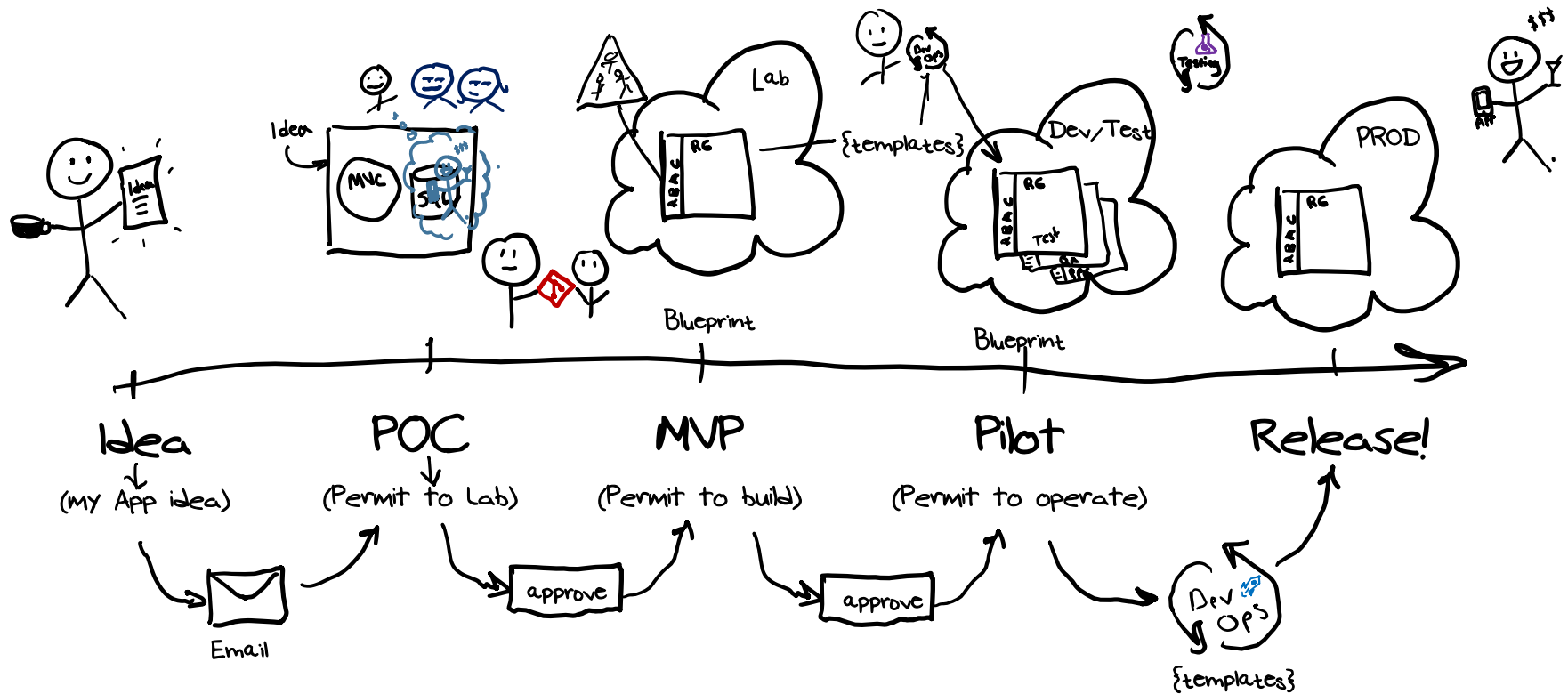


Permit to Cloud = Enterprise{Ready!}



DevOps

Permit to Cloud



Code Collaboration

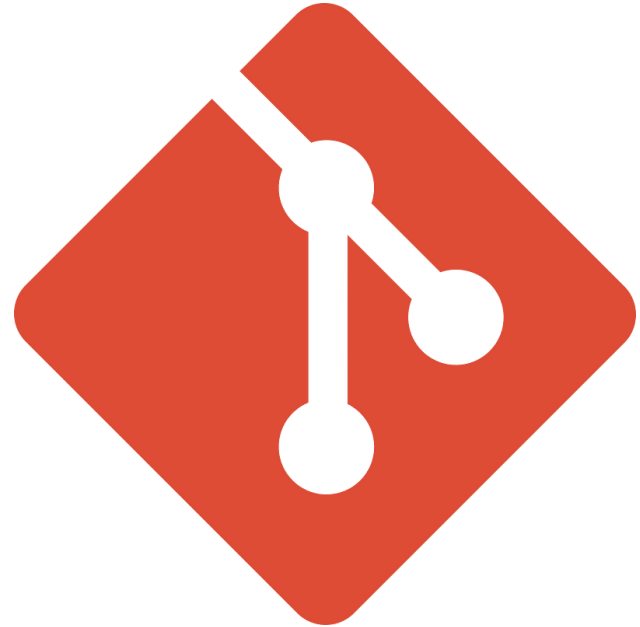
Shared Repositories

Git industry standard

Branching & Forking

Pull Requests to merge code

Versioning



Azure DevOps vs. Github

Azure DevOps

- Microsoft Team Services
- Enterprise focus
- Private repos by default
- One-stop-shop

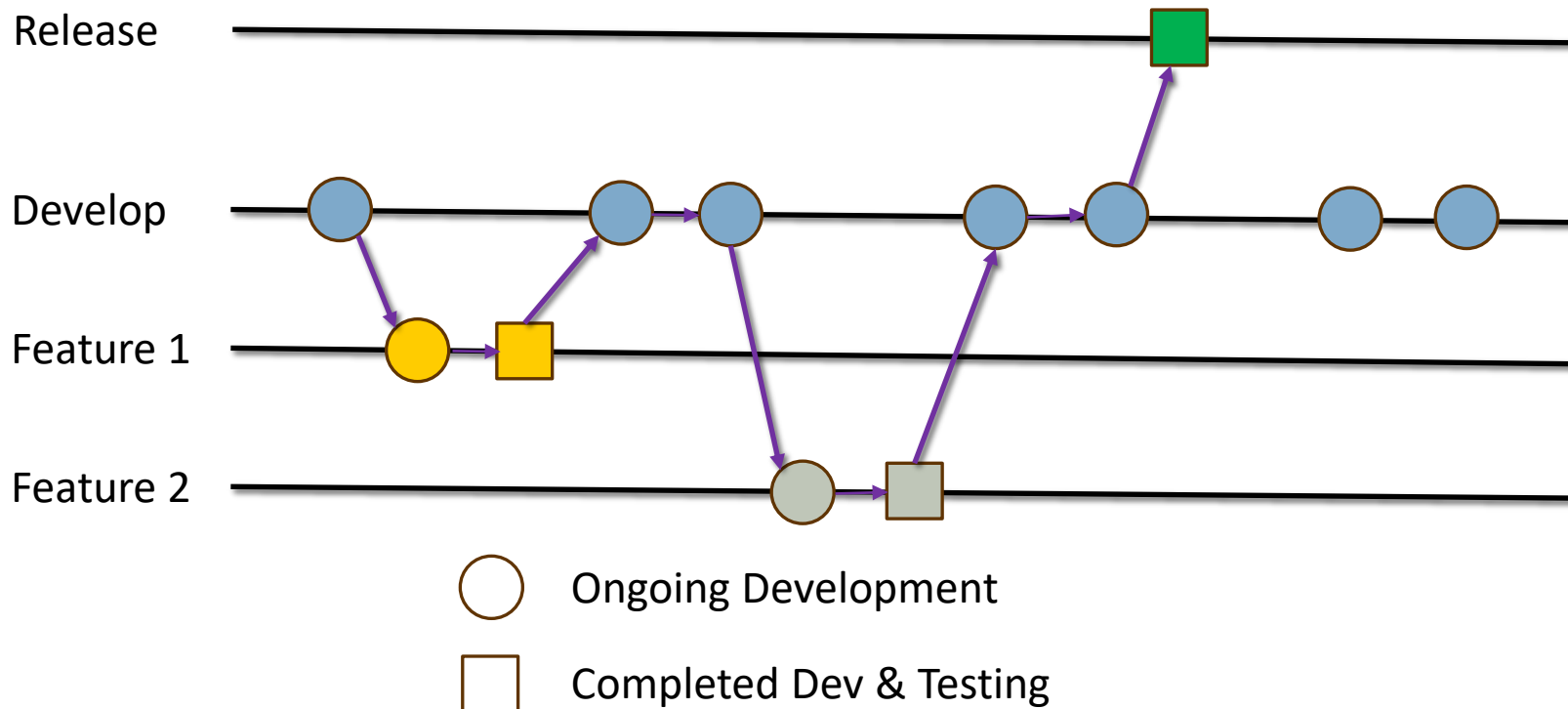
Repos, Boards, Pipelines

Artifacts, Testing

Github

- Open Source favorite
- Community focus
- Public repos by default
- New capabilities
 - Actions
 - Boards

Branching Example: Feature Development



Continuous integration

- Simplifies and automates testing and building of code
- Catch bugs early in development cycle
- Can be scheduled or triggered by code changes
- Produce items known as build artifacts
- Should be fast and easy to run
- Automated running of unit tests

CI / CD

Continuous Integration

- Increases code coverage
- Build faster by splitting build & test runs
- Automatically ensure you don't ship broken code
- Run tests continually

Continuous Delivery

- Automatically deploy code to production
- Ensure deployment targets have latest code
- Use tested code from CI Process

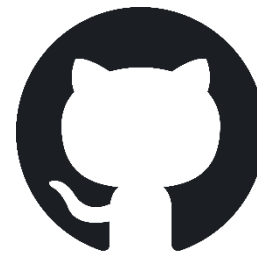
GitHub Actions for CI/CD

Triggered by changes to Code

YAML based syntax

Defines steps to build & deploy

Extendable



DevOps Mindset

What is your branching strategy?

Does it trigger pipelines, if so which ones?

Redeployment of workloads?

What is your Environment Strategy?

Versioning Scheme? Change log? Track changes?

Infrastructure as Code

Database Ops – updates to schema and data?

Release Mindset

What is your branching strategy?

Do you have naming standards for branches?

How do you version things?

Do you have a change log?

Do you link commits to features, bugs, etc in planning tool?

Azure Developer Tool - AZD

Infrastructure + DevOps + Deployment

azd

<https://github.com/azure/azure-dev>

laC

WARNING!



CLICKED TO DEPLOY



Infrastructure as Code (IaC)

- Declarative
- Versioned & managed
- Repeatable
- Descriptive vs Procedural
- Environment drift
- Idempotent
- Current

```
main.bicep M ×
deploy > Bicep > main.bicep > ...
1 | @description('Specifies the location for resources.')
2 | param location string = 'centralus'
3 |
4 | param appName string
5 | param envName string
6 | param color string
7 | @secure()
8 | param secretValue string
9 |
10 | targetScope = 'subscription'
11 |
12 | resource rg 'Microsoft.Resources/resourceGroups@2021-04-01' = {
13 |   name: 'bnk-${appName}-${envName}-rg'
14 |   location: location
15 | }
16 |
17 | module site 'mywebsite.bicep' = {
18 |   scope: resourceGroup(rg.name)
19 |   name: deployment().name
20 |   params: {
21 |     appName: appName
22 |     envName: envName
23 |     secretValue: secretValue
24 |     color: color
25 |   }
26 | }
27 |
28 | output rgName string = rg.name
```

Comparison

Feature	 ARM/Bicep	 Terraform	 Pulumi
Language	JSON + Bicep	HCL/DSL	Code Native, e.g. JavaScript, Python, C#...
Clouds	Azure only	Agnostic + on-prem	Agnostic + on-prem
State Files	Uses Azure Resource Manager natively	Plain-text	Encrypted
Naming standards	Variables & Parameters	Locals	Language native
Environments	Parameter files	Folder structure	Stacks
Preview Changes	az deployment ... what-if	terraform plan	pulumi preview
Infrastructure Cleanup	No	terraform destroy	pulumi destroy
Deployment History	Yes – View in Portal	SCM, TF Cloud*	SCM, Pulumi Enterprise*
Code Reuse	Hosted JSON URIs	Modules + Registry	Code native packages, NPM

* refers to a premium feature from vendor, i.e. Terraform Cloud or Pulumi Enterprise

Source: <https://julie.io/writing/arm-terraform-pulumi-infra-as-code>



Project Bicep

- ARM Transpiler, generates ARM as output
- Simpler syntax reduces complexity of ARM
- Modularity
- Support for all resource types and API versions
- A domain-specific-language for Azure
- No state or state files to manage
- No cost, open source



Bicep File

```
targetScope = '<scope>'

@<decorator>(<argument>)
param <parameter-name> <parameter-data-type> = <default-value>

var <variable-name> = <variable-value>

resource <resource-symbolic-name> '<resource-type>@<api-version>' = {
  <resource-properties>
}

module <module-symbolic-name> '<path-to-file>' = {
  name: '<linked-deployment-name>'
  params: {
    <parameter-names-and-values>
  }
}
```

```
output <output-name> <output-data-type> = <output-value>
```



Terraform

Created by **Hashicorp**

HCL Language

Multi-cloud, provider based

Tool for versioning infrastructure

Uses **state** information for execution plan

Install is simple download of the executable and run



Terraform Example: App Service Plan

```
resource "azurerm_app_service_plan" "plan" {  
  name                = local.host_name  
  location             = "${azurerm_resource_group.main.location}"  
  resource_group_name = "${azurerm_resource_group.main.name}"  
  
  sku {  
    tier = "Free"  
    size = "F1"  
  }  
}
```

Cloud App – Day 1: POC

Sandbox Subscription

- ☐ Visual Studio Benefits

Services

- ☐ Hosting Plan
- ☐ App Services
 - ☐ *Click to deploy*



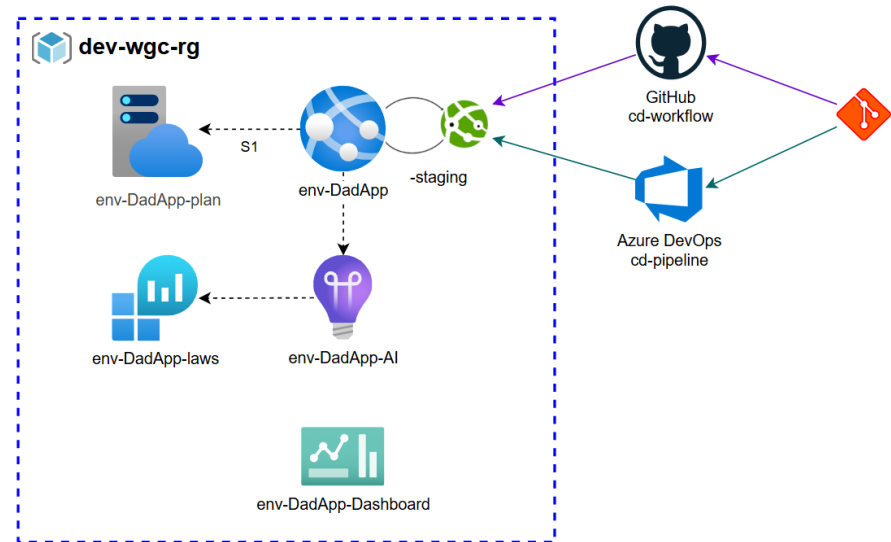
Cloud App – Day 2: Collaboration Env

Shared Subscription

- ## ❑ Pay as You Go

Services

- ☐ Hosting Plan
- ☐ App Services
- ☐ Git Collaboration
 - ☐ Continuous Deployment
- ☐ IaC
 - ☐ Staging Slot
 - ☐ App Insights
 - ☐ Log Analytics Workspace
 - ☐ Dashboard



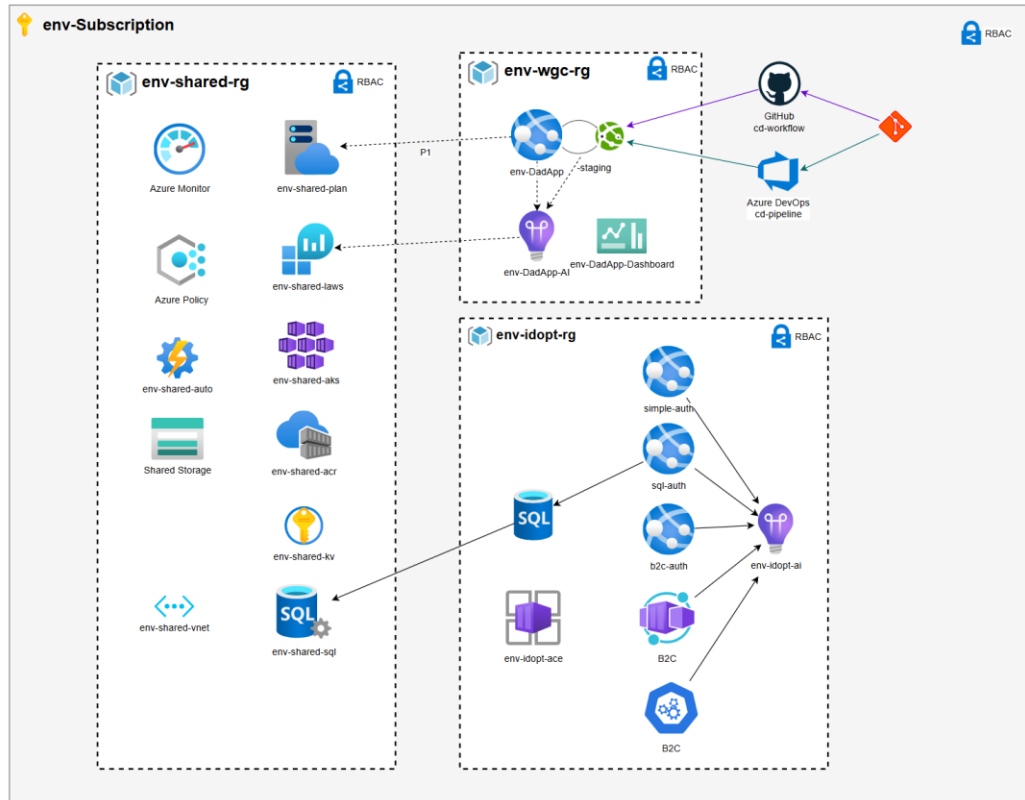
Cloud App – Day 3: Shared Env

Shared Resources

- ☐ Compute
- ☐ Storage
- ☐ Secrets
- ☐ Messaging
- ☐ Automation

Workload Resource Groups

- ☐ Naming & Tagging
- ☐ RBAC Assigned Access
- ☐ Service Principals
- ☐ Infrastructure as Code
- ☐ Locks
- ☐ Resiliency



What makes a Cloud **well governed**?

It is **Secure**, my data and compute are protected from attackers

It is **Reliable**, in that it runs and is available when and where I need it

It is **Efficient**, I'm not paying more than I should for the services I use

It is **Operational**, I know enough to keep it running

Well
Governed
Cloud
Traits

Predictable and Efficient
Business Alignment
Developer Empowerment
Velocity with Safety
Consistency at Scale
Observable & Automatic
Secure & Resilient

The Well Run Cloud

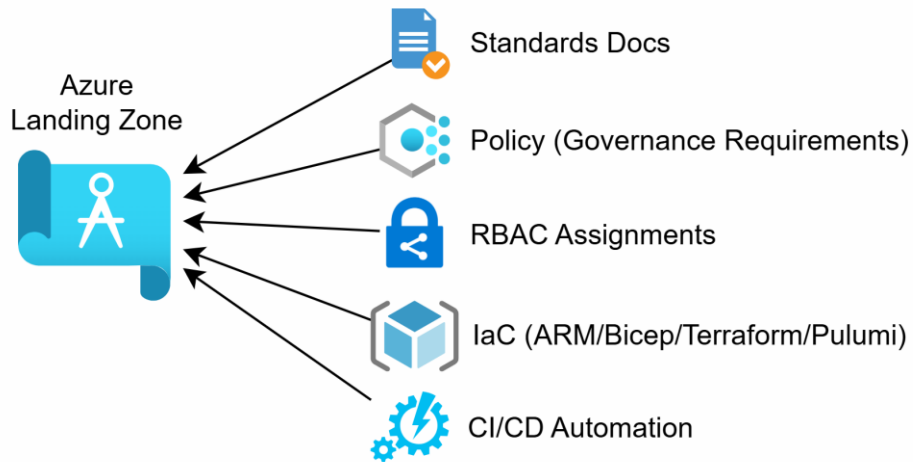
Ground rules for Cloud Adoption

Documented & Socialized

Current with Tooling

Flexible

Evolving with Organization



Tactics

Code & Version Control

Environments & Releases

Automation & Testing

Security & Secrets

Monitoring & Rollback

Documentation & Process Visibility

Cloud Governance Concerns



Azure and Github	-	https://bit.ly/azGitHub
Azure DevOps	-	https://bit.ly/azDevOps
Best Practices	-	https://bit.ly/azBestPractices
Cost Management	-	https://bit.ly/azCostMgmt
Tagging	-	https://bit.ly/azTagging
Location Selection	-	https://bit.ly/azRegions
Naming Standards	-	https://bit.ly/azNames



Naming Standards

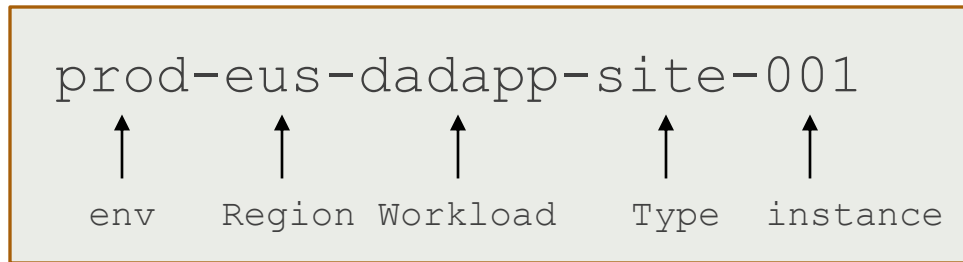
Set standards early

Define a structure that reflects your needs

Use prefixes and suffixes and hyphens (where allowed)

A good name will answer questions like workload, prod or non-prod, service type, and where it's running?

Example:



Subscription Strategy

Separate production workloads from non-Prod

Use Azure Dev/Test offer for nonproduction

Organize with Management Groups

Use multiple subscriptions as needed

Azure Migration enables movement of services between subscriptions

Tagging Strategy

Proper tagging simplifies reporting

Azure supports up to 15 tags per resource

Answer questions like environment, workload, data classification, business unit, cost center, date created

Include tags when defining templates

Location selection

60+ announced regions available

General rule is keep compute and storage together

Confirm services you're using are offered

Use Region pairing for Disaster Recovery location

Portal has list of products available by region

Cost Management

Tagging is key to reporting and cost management

Leverage existing license agreements

Use Dev/Test for non-production workloads

Right size services to meet your requirements

Delete or shut down unused resources

Use multiple small instances vs single larger one

Enable automated scaling where you can

Policy and Compliance

Use policies to define requirements for environment

Policy is JSON definition that is evaluated

Actions effects include deny, audit, modify, etc...

Initiative is collection of policies

Built in initiatives include: NIST, IRS1075, SWIFT, ISO27001, HIPAA, and many more

Hallmarks of a Well Governed Cloud

Documentation

Reference Architecture

Collaboration process

Versioning

Release strategy

Automation

Resource Organization

Access Management

Subscription Strategy

Cost Optimization

Unified Logging

Locks

**Well
Governed
Cloud**

Docs

Tools

Resource Consistency

Security

DevOps mindset

Efficient & Available - Resilient

Operations

Landing Zones

Cloud Adoption Framework

How do we plan and start the journey to the cloud?

What do we need to know to get started?

How do we measure success?

What should we expect on the way?

How do we get there?

**Ready
Plan
Adopt**

<https://bit.ly/wgcCAF>

Permit to Cloud: Runway Checklist

Blueprint factory and module registry

CI/CD pipelines for infra + apps

Guardrails: policies & tests

Monitoring and self-healing



Access Strategy

How do I get access to your cloud?

Who has access to deployments?

Individual vs Security Group grants?

Multi-Factor Authentication enabled?

Using Privileged Identity Management (PIM)?

Policy and Compliance

Use policies to define requirements for environment

Policy is JSON definition that is evaluated

Actions effects include deny, audit, modify, etc...

Initiative is collection of policies

Built in initiatives include: NIST, IRS1075, SWIFT, ISO27001, HIPAA, and many more

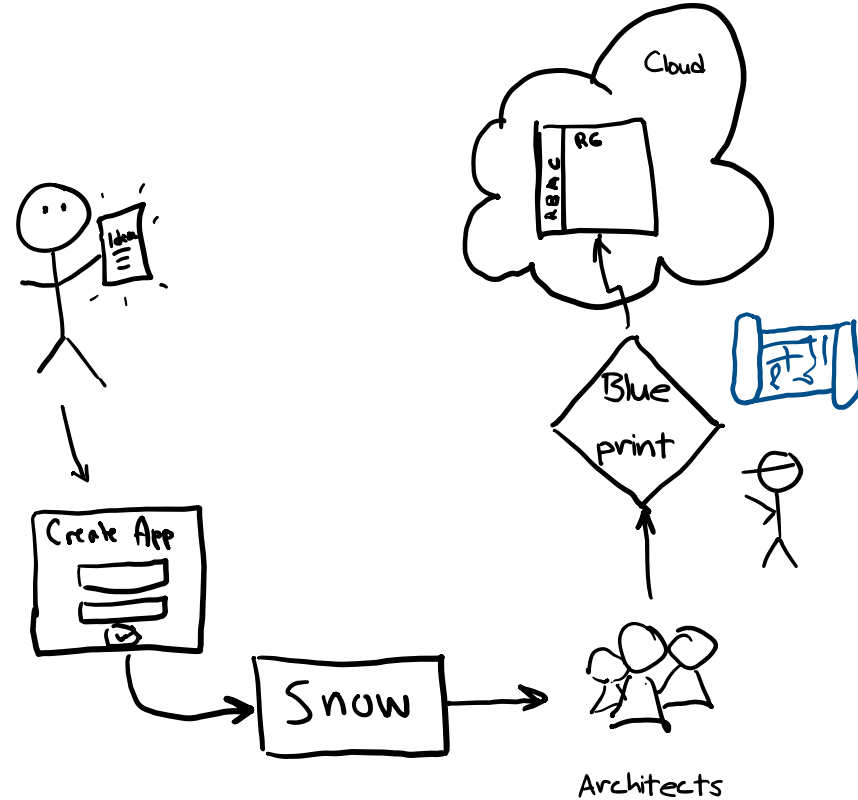
Azure Blueprints

Resource Group

Access Control (RBAC)

Policies

Templates



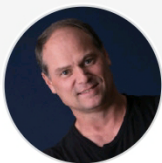
Well Governed Cloud



Doing things right

is not the same as

Doing the right things



Mike Benkovich

Enterprise Cloud Architect,
Consultant, Developer Tools Ev...



Call to Action...

Where can I get more info?

Give me feedback on LinkedIn
(Scan the QR Code to the left)

Visit my blog www.benkotips.com

Azure Office Hour Fridays!
<https://bit.ly/BnkAzHrs>

Try it out with **low hanging fruit**

What is your **Environment** Strategy?

Production **separated** from non-production

Mimics production behavior, but **safe**

Development is **ephemeral**, can be rebuilt as needed

Promotion between environments to ensure functionality and performance and behaviors

Access is **monitored**, even **audited**

Leads to **Subscription** strategy

Service Selection

What level of service do you need?

Compute, Data, Messaging, Networking

Lift & Shift vs Architected for the Cloud

Resiliency Plan

What region(s) are you deploying to?

What resilience features are you enabling?

Backup & restore, documented, tested and practiced?

Upgrading and Patching schedule?

Service update awareness and actions?

Observability

Logging strategy

Reporting

Dashboards

Alerting strategy

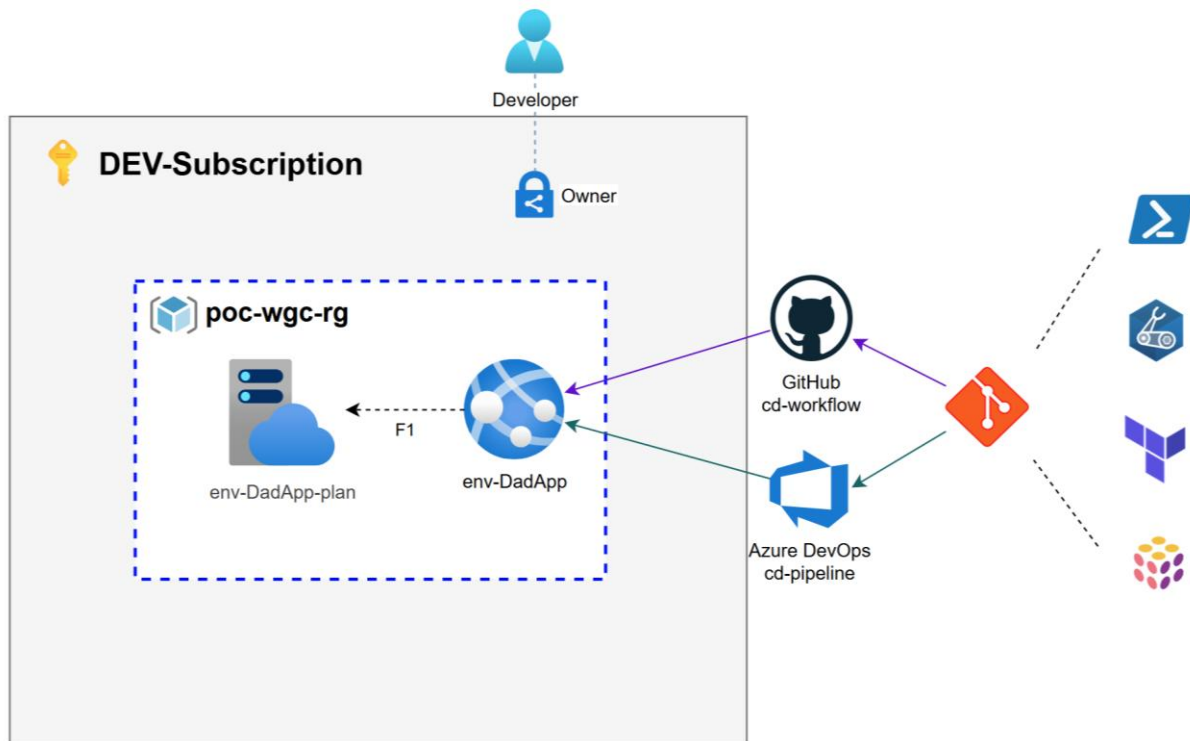
Permit to Build

Dev Pattern

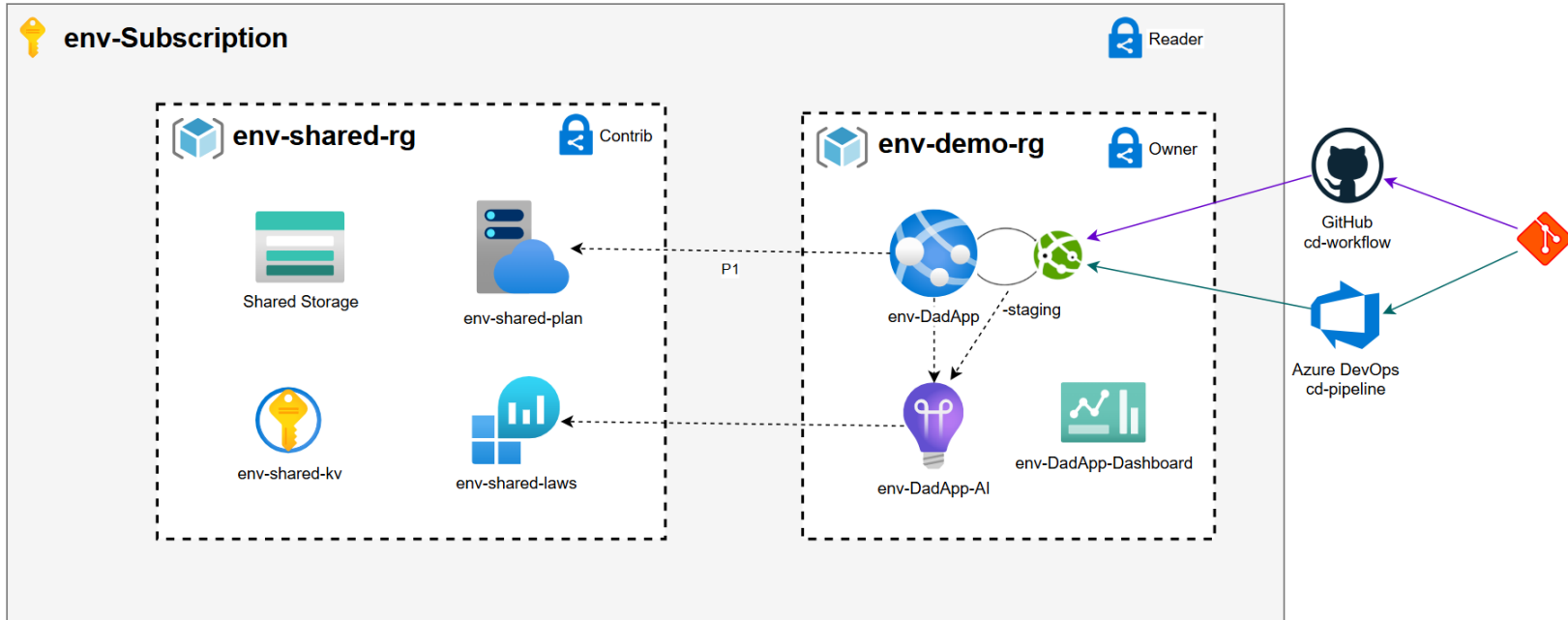
Git Repo

IaC

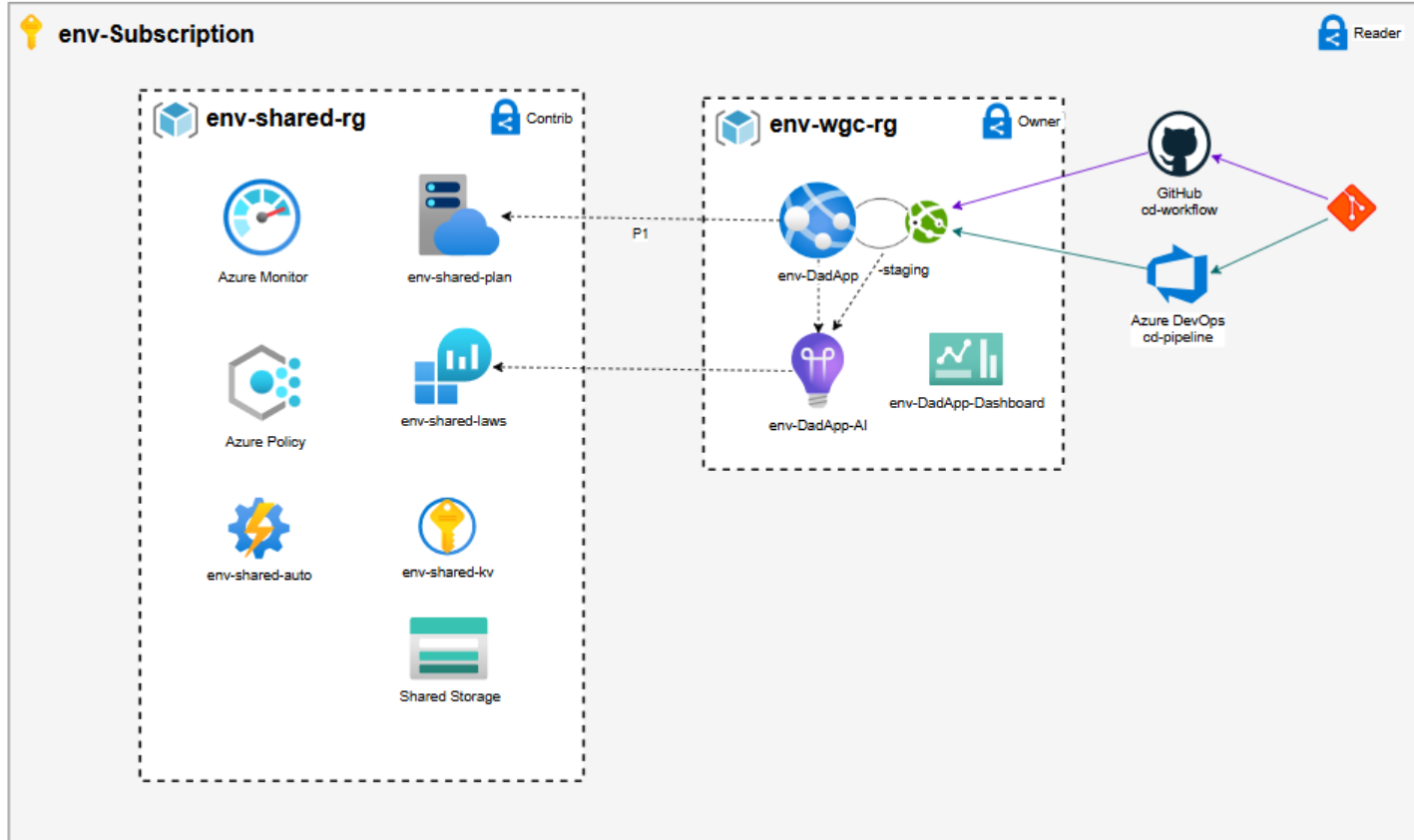
DevOps CI/CD



Sharing Resources – Hosting & Keyvaults



Sharing Resources - Monitoring & Policy



Takeaways from today

Enable Architectural **Evolution**

Make **Data** Driven Decisions

Educate and **Enable** your Team

Automation is your friend